EPFL - Computer Science

CS-550 Formal Verification

Lecture 04 — October 28, 2026

Author: Jonathan Dutoit

Professor: Viktor Kuncak

Automating First-Order Logic Profs Using Resolution

First-order logic formula:

$$\begin{aligned}&[(\forall x.\exists y.R(x,y)) \wedge \\ &(\forall x.\forall y.(R(x,y) \to \forall z.R(x,f(y,z)))) \wedge \\ &(\forall x.(P(x) \vee P(f(x,a))))] \\ &\to \forall x.\exists y.(R(x,y) \wedge P(y))\end{aligned}$$

First-order logic supports arbitrary (uninterpreted) predicates and functions. Their meaning will be constrained through axioms. Note that it contains:

- propositional operations $\wedge, \vee, \neg, \to$
- variables *x, y*
- function symbols and constants: *f, a*
- predicate symbols: *P, R*
- quantifier $\forall, \exists$ (only over variables, not function or predicate symbols - FO)

FOL can be used in practice for formalize most of mathematics (e.g., through set theory axioms), and thus all of **program verification problems**. To prove whether a property holds, we can proceed as follows:

1. Describe the property using a formula F
2. Describe the functions and relations in F using a sequence of axioms S
3. Check if the sequence $(\neg F; S)$ is contradictory. If yes, then F follows from S.

Definition. A first-order *signature* $\mathcal{L}$ specifies a countable set of function symbols f (constants c are functions symbols taking no arguments), and predicate symbols p. Syntax of formulas *(F)* and terms *(t)* in first-order logic:

$$F ::= p(t_1, ..., t_n) \,|\, \forall x.F \,|\, \exists x.F \,|\, \top \,|\, \bot \,|\, \neg F \,|\, F_1 \wedge F_2 \,|\, F_1 \vee F_2 \,|\, F_1 \to F_2 \,|F_1 \leftrightarrow F_2$$

$$t ::= x \,|\, c \,|\, f(t_1, ..., t_n)$$

Where we assume that the set of variables $x \in Var$, predicate symbols p and function symbols f are all *countably infinite* sets.

Arity: *ar* denotes arity of functions and predicate symbols which defines how many arguments it takes. For instance, *ar(f) = 2* means f takes two arguments, so it is allowed to form a term $f(t_1, t_2)$, and also *ar(p) = 2* for predicate symbol means that it is allowed to form formula $p(t_1, t_2)$.

We interpret each f as $\bar{f} : D^{\text{ar(f)}} \to D$ and each p as $\bar{p} : D^{\text{ar(p)}} \to \{0, 1\}$ (or: $\subseteq D^{\text{ar(p)}}$)

- *atomic formula*: $p(t_1, ..., t_n)$ (contains no logical connective or quantifier)
- *literal*: either an atomic formula, or its negation $\neg p(t_1, ..., t_n)$
- *clause*: a disjunction of literal, e.g., $\neg p(x, f(y)) \vee q(y) \vee \neg r(x, z)$

Ex.: signature $\mathcal{L} = \{P, R, a, f\}$ where

- P is a predicate symbol, *ar(P) = 1*
- R is a predicate symbol, *ar(R) = 2*
- a is a constant
- f is a function symbol, *ar(f) = 2*

An **interpretation** for this language is any structure (*D, e*) where:

- D $\neq \emptyset$ (*D stands for domain*, it can be finite or infinite)
- $e(P) : D \to \{0, 1\}$
- e(R): $\text{D}^2 \to \{0, 1\}$
- e(a) $\in D$
- e(f) : $\text{D}^2 \to D$

Considering the following interpretation:

- $D = \{0, 1, 2...\}$ (non-negative integers)
- $e(P)(n) =$ if $(2 \mid n)$ then 1 else 0
- $e(R)(x, y) =$ if $(x \leq y)$ then 1 else 0
- *e(a) = 1*
- $e(f)(x, y) = x + y$

We can then talk about the truth value of any closed formula, e.g.:

text, or tree	true or false
$(\forall x.\exists y.R(x, y)) \wedge$	$(\forall x.\exists y.x \leq y)$
$(\forall x.\forall y.(R(x, y) \to \forall z.R(x, f(y, z)))) \wedge$	$(\forall x.\forall y.x \leq y \to \forall z.x \leq (y + z)) \wedge$
$(\forall x.(P(x) \vee P(f(x, a))))]$	$(\forall x.(2 \mid x) \vee (2 \mid x + 1))$
$\to \forall x.\exists y.(R(x, y) \wedge P(y))$	$\to \forall x.\exists y.(x \leq y \wedge (2 \mid y)$

Semantics in general: interpreter written in set theory

Definition. A first-order *interpretation* is $I = (D,\ e)$ where $D \neq \emptyset$ and e maps constants, function and predicate symbols as follows:

- each predicate symbol p with $ar(p) = n$ into $e(p) : D^n \to \{0, 1\}$
- each function symbol f with $ar(f) = n$ into a total function of n arguments, $e(f) : D^n \to D$ (and each constant c into element of D, i.e., $e(c) \in D$)
- maps each variable x to element of D, i.e., $e(x) \in D$

We then define $[\![F]\!]_I \in \{0, 1\}$ to denote whether F is true (1) or false (0) in interpretation I. The rules are the expected rules *recursive* on the formula tree:

$$[\![\ \mathrm{F}_1 \wedge F_2]\!]_I = [\![F_1]\!]_I \wedge [\![F_2]\!]_I, \quad [\![F_1 \vee F_2]\!]_I = [\![F_1]\!]_I \vee [\![F_2]\!]_I, \quad [\![\neg F]\!]_I = \neg [\![F]\!]_I, \quad [\![\bot]\!]_I = 0$$

$$[\![\ \mathrm{f}(\mathrm{t}_1, \ldots, t_n)]\!]_I = e(f)([\![t_1]\!]_I, \ldots, [\![t_n]\!]_I), \quad [\![p(t_1, \ldots, t_n)]\!]_I = e(p)([\![t_1]\!]_I, \ldots, [\![t_n]\!]_I), \quad [\![x]\!]_I = e(x)$$

$$[\![\forall x.F]\!]_{(D,e)} = \text{if } (\forall d \in D.([\![F]\!]_{(D,e[x:=d])} = 1)) \text{ then 1 else 0}$$
$$[\![\exists x.F]\!]_{(D,e)} = \text{if } (\exists d \in D.([\![F]\!]_{(D,e[x:=d])} = 1)) \text{ then 1 else 0}$$

.

where $e[x := d](y) = (\text{if } x = y \text{ then } d \text{ else } e(y))$

Note: we *cannot* write a FOL formula that quantifies over function and relation symbols (that would be *second* or, generally, *higher* order).

To make statements that *do not depend* on a particular interpretation, we use concepts of **satisfiability** and **validity**. Let's use the FOL formula $\forall x.\exists y.(p(x, y) \wedge q(y, x))$

Definition. A formula F is **satisfiable** if **there exists an interpretation** $(D,\ e)$ such that $[\![F]\!]_{(D,e)} = 1$. Which is equivalent to an informal statement:

$$\exists \mathbf{D} \neq \emptyset.\exists \bar{\mathbf{p}}, \bar{\mathbf{q}} : \mathbf{D}^2 \to \{0, 1\}.\forall x \in D.\exists y \in D.((\bar{p}(x, y) \wedge \bar{q}(y, x)) = 1)$$

The domain, functions, and relations are all *existentially* quantified.

Definition. A formula F is **valid** if **for all interpretations** $(D,\ e)$ (for arbitrarily small or large sets D and all possible choices of e), $[\![F]\!]_{D(e)} = 1$. Which is equivalent a mathematical statement

$$\forall \mathbf{D} \neq \emptyset.\forall \bar{\mathbf{p}}, \bar{\mathbf{q}} : \mathbf{D}^2 \to \{0, 1\}.\forall x \in D.\exists y \in D.((\bar{p}(x, y) \wedge \bar{q}(y, x)) = 1)$$

The domain, functions. and relations are all *universally* quantified.

Observation: F is valid if and only if $\neg F$ is not satisfiable. We will be checking satisfiability (aiming for a negative answer).

Negation normal form for FOL

To check the validity of formula F, we check the satisfiability of $\neg F$. In general, we will transform $\neg F$ into a *normal form*.

Observation If $F \leftrightarrow G$ is a valid FOL formula, then inside any other FOL formula H we can replace a sub-formula F with G without changing the truth value of the formula: $H[F] \rightsquigarrow H[G]$.

We can transform formulas to negation normal using transformations such as these:

$$\begin{aligned} F_1 \leftrightarrow F_2 &\rightsquigarrow (F_1 \rightarrow F_2) \wedge (F_2 \rightarrow F_1) \\ F_1 \rightarrow F_2 &\rightsquigarrow \neg F_1 \vee F_2 \\ \neg\neg F &\rightsquigarrow F \\ \neg(F_1 \wedge F_2) &\rightsquigarrow \neg F_1 \vee \neg F_2 \\ \neg(F_1 \vee F_2) &\rightsquigarrow \neg F_1 \wedge \neg F_2 \\ \neg\forall x.F &\rightsquigarrow \exists x.\neg F \\ \neg\exists x.F &\rightsquigarrow \forall x.\neg F \end{aligned} \quad .$$

In NNF, negation applies only to atomic formulas (node leaves) and the only other propositional connective are $\wedge, \vee$.

Skolem function

Definition. *Skolemization* replaces an existentially quantified variable ($\exists y$) with a function that chooses a suitable value for y. The function takes as arguments the variables that the choices depends on.
Suppose we have:

$$\forall x \exists y p(x, y)$$

We don't know which y works for each x, so we introduce a new function *s(x)* that chooses a suitable one. The formula becomes

$$\boxed{\forall x p(x, s(x))}$$

Here, s is called a **skolem function**. Skolemization **preserves satisfiability**, but it does not necessarily produce a logically equivalent formula.

A skolem function depends on the *universally quantified variables* that appear before the existential quantifier.

- No preceding universal variables $\exists y p(y) \rightarrow \boxed{p(c)}$ replace y with a constant c.
- Two preceding universal variables $\forall x \forall z \exists y p(x, z, y) \rightarrow \boxed{\forall x \forall z p(x, z, s(x, z))}$ replace y with *s(x, z)*
- More than one quantifier $\forall x \exists y \forall z \exists w p(x, y, z, w)$, then we eliminate existential quantifiers **from left to right** $\rightarrow \forall x \forall z \exists w p(x, f(x), z, w) \rightarrow \boxed{\forall x \forall z p(x, f(x), z, g(x, z))}$

Example with FOL

$$\begin{aligned} \neg\,(((\forall x.\exists y.R(x,y)) \;\wedge \qquad & \neg(a \to b) \rightsquigarrow a \wedge \neg b \\ (\forall x.\forall y.(R(x,y) \to \forall z.R(x,f(y,z)))) \;\wedge \qquad & a \to b \rightsquigarrow \neg a \vee b \\ (\forall x.(P(x) \vee P(f(x,a)))) & \\ \to \forall x.\exists y.(R(x,y) \wedge P(y)) & \end{aligned}$$

since $\neg(a \to b) \leftrightarrow \neg(\neg a \vee b) \leftrightarrow a \wedge \neg b$, it becomes

$$\begin{aligned} (((\forall x.\exists y.R(x,y)) \;\wedge \qquad & y \rightsquigarrow s_1(x) \\ (\forall x.\forall y.(\neg R(x,y) \vee \forall z.R(x,f(y,z)))) \wedge & \\ (\forall x.(P(x) \vee P(f(x,a))))\wedge & \\ (\exists x.\forall y.(\neg R(x,y) \vee \neg P(y))) \qquad & x \rightsquigarrow s_2 \end{aligned}$$

by skolemization it finally becomes:

$$\begin{aligned} (\forall x.R(x,s_1(x))) \wedge & \\ (\forall x.\forall y.(\neg R(x,y) \vee \forall z.R(x,f(y,z)))) \wedge & \\ (\forall x.(P(x) \vee P(f(x,a))))\wedge & \\ (\forall y.(\neg R(s_2,y) \vee \neg P(y))) & \end{aligned}$$

The resulting formula only has *universally quantified variables*.

Moving quantifiers and conjunctive normal form

Note that all bound quantified variables can be renamed while preserving equivalence, and that a quantifier can be pulled in or out of a formula where it does not occur:

- $(\forall x.F) \leftrightarrow \forall y.F[x := y]$, where y is fresh
- $(\forall x.F) \vee G \leftrightarrow \forall x.(F \vee G)$
- $(\forall x.F) \wedge G \leftrightarrow \forall x.(F \wedge G)$

We can thus pull quantifiers outwards. If we choose, we can obtain: **prenex normal form**, a formula of the form

$$Q_1 x_1.Q_2 x_2. \ldots Q_n x_n.G$$

where $Q_i \in \{\forall, \exists\}$ and G has no quantifiers.

We can then transform G to **conjunctive normal form**, as for propositional logic:

$$(F \wedge G) \vee H \leftrightarrow (F \vee H) \wedge (G \vee H)$$

We can also introduce *fresh predicate symbols* to avoid exponential blowup.

Conjunction of clauses

> **Theorem.** Each formula F can be transformed into an *equisatisfiable* formula of the form:
>
> $$(\forall x_1, \ldots, x_n.C_1) \wedge \cdots \wedge (\forall x_1, \ldots, x_n.C_m)$$
>
> where each C_i is quantifier-free and, moreover, a disjunction of first-order literals.

Note: a first-order *literal* is either an atomic formula, that is $p(t_1, \ldots, t_k)$ where p is a predicate symbol, or a negation of an atomic formula, $\neg p(t_1, \ldots, t_k)$.

Indeed, we have seen how to move $\neg$ to predicates (*NNF*), eliminate $\exists$ (skolemization), and pull $\forall$ and $\wedge$ to the outside, leaving $\vee$ applied to literals. Each C_i is a (first-order) **clause**. We can also view it as a *set*, because the order of disjuncts does not matter up to equivalence. A clause that is an *empty set* means $\bot$.

For a given formula F, denote the set of obtained clauses $\{C_1, \ldots, C_m\}$ by clauses(F). We *omit* universal quantifiers because all variables are universally quantified. We use a convention to denote variables by *x, y, z, ...* and constants by *a, b, c, ...*

Sets of formulas when checking satisfiability

Definition. A *set* of formulas S is *true* in an interpretation I if *every* formula is true in that interpretation:

$$[\![S]\!]_I = \text{if } (\forall F \in S. [\![F]\!]_I = 1) \text{ then 1 else 0}$$

If S is finite, then $[\![S]\!]_I = [\![\bigwedge_{F \in S} F]\!]_I$.
We say two sets are *equivalent* iff the sets of interpretations in which they are true are equal.
We say that S is *satisfiable* iff there exists I such that $[\![S]\!]_I = 1$.

Thus, F is satisfiable iff clauses(F) is satisfiable.

Clauses for the example

The skolemized formula

$$\begin{aligned}(\forall x. R(x, s_1(x))) \wedge \\ (\forall x. \forall y. (\neg R(x, y) \vee \forall z. R(x, f(y, z)))) \wedge \\ (\forall x. (P(x) \vee P(f(x, a)))) \wedge \\ (\forall y. (\neg R(s_2, y) \vee \neg P(y)))\end{aligned}$$

gives the set of clauses:

$$\begin{aligned}&R(x, s_1(x)),\\ &\neg R(x, y) \vee R(x, f(y, z)),\\ &P(x) \vee P(f(x, a)),\\ &\neg R(s_2, y) \vee \neg P(y)\end{aligned}$$

or, if we represent clauses themselves as sets:

$$\{R(x, s_1(x))\}, \quad \{\neg R(x, y), R(x, f(y, z))\}, \quad \{P(x), P(f(x, a))\}, \quad \{\neg R(s_2, y), \neg P(y)\}$$

Applying sound inference rules

Definition. We say that an inference rule

$$\frac{F_1 \quad \ldots \quad F_n}{G}$$

is **sound** iff the formula $(F_1 \wedge \cdots \wedge F_n) \rightarrow G$ is valid.

Observation. Let $\frac{F_1 \quad \dots \quad F_n}{G}$ be a sound inference rule with $F_1, \dots, F_n \in S$. Then S and $S \cup \{G\}$ are equivalent.

Ex.: using the convention that variables are $\forall$-quantified, the *instantiation* rule

$$\frac{C}{C[x := t]}$$

is sound, where $C[x := t]$ denotes substituting variables of C by some terms. In general, t may contain other variables. Special cases of instantiation are:

- *renaming*: replace variables with fresh variables
- *ground instantiation*: replace variables with ground terms, that have no variables (built only from constants and function symbols)

Resolution for FOL clauses

The following formula (transitivity of universal implication) is easily proven to be *valid* for all formulas *F, G, H*, where x denotes the finite list of all free variables in *F, G, H*:

$$(\forall x.(F \to G)) \land (\forall x.(G \to H)) \to (\forall x.(F \to H))$$

Write it down as an inference rule, assuming that all formulas are universally quantified:

$$\frac{F \to G \qquad G \to H}{F \to H}$$

Expressing $\to$ using $\lor$ and $\neg$:

$$\frac{\neg F \lor G \qquad \neg G \lor H}{\neg F \lor H}$$

Let G be a literal L, let $\neg F$ be equivalent to clause C_1 and H to clause C_2. We get:

$$\frac{C_1 \lor L \qquad \neg L \lor C_2}{C_1 \lor C_2}$$

resolution (simple, without unification)

Resolution with instantiation

We call literals L and $\neg L$ **complementary**.

If literals are not complementary, we may still be able to apply the substitution rule to make them complementary:

$$\frac{C_1 \lor L \qquad \neg L \lor C_2}{C_1 \lor C_2} \quad \rightsquigarrow \quad \frac{C_1' \lor L' \qquad \neg L'' \lor C_2''}{C_1 \lor C_2}$$

To check whether we can find a substitution to make L' and L'' identical, we can use (syntactic, first-order) **unification**, which has a *linear-time* algorithm.

We rename variables in the two clauses so that they are disjoint, then instantiate them by computing the *most general unifier*, so that literals become complementary (one is an atomic formula and the other its negation).

Applying resolution

Apply resolution and instantiation to the clauses in our example:

1. $R(x, s_1(x))$
2. $\neg R(x, y) \vee R(x, f(y, z))$
3. $P(x) \vee P(f(x, a))$
4. $\neg R(s_2, y) \vee \neg P(y)$
5. $(1, 2) : R(x, f(s_1(x), z))$ $\quad y \rightsquigarrow s_1(x)$
6. $(1, 4) : \neg P(s_1(s_2))$ $\quad x \rightsquigarrow s_2,\ y \rightsquigarrow s_1(x)$
7. $(3, 6) : P(f(s_1(s_2), a))$ $\quad x \rightsquigarrow s_1(s_2)$
8. $(4, 7) : \neg R(s_2, f(s_1(s_2), a))$ $\quad y \rightsquigarrow f(s_1(s_2), a)$
9. $(5, 8) : \bot$ $\quad x \rightsquigarrow s_2,\ z \rightsquigarrow a$

We can derive $\bot$ from the set of clauses. Thus, the set of clauses is *unsatisfiable*. The formula we started from as well.

Conclusion of the procedure on this example

Thus, the formula before negation:

$$
\begin{aligned}
&[(\forall x.\exists y.R(x, y)) \wedge \\
&(\forall x.\forall y.(R(x, y) \rightarrow \forall z.R(x, f(y, z)))) \wedge \\
&(\forall x.(P(x) \vee P(f(x, a))))] \\
&\rightarrow \forall x.\exists y.(R(x, y) \wedge P(y))
\end{aligned}
$$

was **valid**. We have proven that this formula holds in *all possible models* (no matter how small, large, or complex they might be).

Remarkably, *resolution with instantiation* is **refutationally complete**: if the formula is valid, then from the clauses equisatisfiable to its negation we will find a contradiction.